Introduction to API Security

API (Application Programming Interface) is defined as a set of rules and protocols that enables different software applications to communicate with each other. It basically acts as the messenger (or the middleman) handling the requests and responses between client (the application making the requests) and server (the application or system that provides data or service to the client).

In the context of the MoMo SMS application that we are working on, API serves as the bridge between the raw XML SMS data and the client applications like web or mobile applications that need access to that data.

API security is a practice of protecting the APIs from attacks, unauthorized access, and data breaches, it involves establishing different strategies and controls throughout the entire API lifecycle to ensure the confidentiality, integrity, and availability of the data and services they connect.

Authentication vs Authorization
Authentication is a process of identifying if the user is who they claim to be, by receiving and processing their credential information to see if they match the ones stored on the server.
Authorization is a process of determining what the user is allowed to access in the system and what they are not allowed to.

In the MoMo SMS app, we employed the use of authentication to identify who the user is, by reading their credentials from the request header using the method called “Basic Authentication” in Python, to make sure that only users with the correct credentials can access SMS data.

API Endpoints Documentation

Authentication Requirement

- **Type:** HTTP Basic Authentication.
- **Required Header:** `Authorization: Basic <base64_encoded_credentials>`.

Endpoint: `GET /transactions`

Description: Retrieves a list of all parsed SMS transaction records.
Response (200 OK): A JSON list of transaction objects.
Error (401 Unauthorized): Missing or incorrect credentials.

Endpoint: `GET /transactions/{id}`

Description: Retrieves a single transaction by its unique ID.
DSA Note: This endpoint utilizes the **Dictionary Lookup** for O(1) efficiency in production, though we also tested it with **Linear Search** for comparison.
Error (404 Not Found): The ID does not exist in the dataset.

Endpoint: `POST /transactions`

Description: Adds a new SMS record to the system.
Request Body: JSON object containing `type`, `amount`, `sender`, `phone` and `from`
Response (201 Created): Returns the newly created object with its assigned ID.

Endpoint: `PUT /transactions/{id}`

Description: Updates an existing transaction's details.
Request Body: JSON containing only the fields to be updated (e.g., `{"amount": "2500"}`).
Response (200 OK): `{"message": "Transaction updated successfully"}`.
Error (400 Bad Request): Malformed JSON or invalid data types.

Endpoint: `DELETE /transactions/{id}`

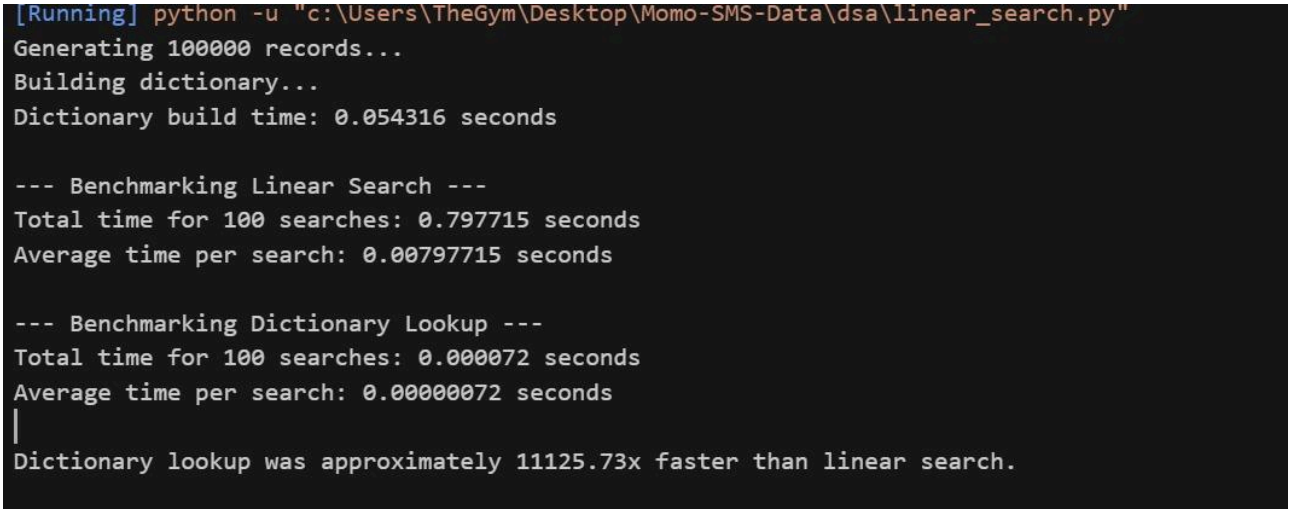
Description: Removes a transaction record from the system.
Response (204 No Content): Successful deletion with no body returned.

Results of DSA Comparisons

We run a simple test to measure the efficiency of doing a linear vs. dictionary lookup.
We generate 100,000 records and attempt to search for 1 record out of it using both linear and dictionary lookup. We then measure how much time each takes and see the difference between both.

This is the logs of executing the test script. The numbers may slightly differ as you run the script multiple times because we added a bit of randomness but the difference shows in each instance.

Terminal logs:



As you can see, linear search took about 0.797715 seconds per 100 searches while dictionary look up took about 0.000072 seconds. Leading to a conclusion that dictionary lookup was about **11,079.38** faster.

Reflection on Basic Authentication

Basic Authentication is an HTTP authentication method that uses a combination of a username and password to verify a user's identity. These credentials are **encoded** using **Base64** and sent in the HTTP request header.

For example, if your username is **admin** and password is **password123**, the encoded string would look like this:

```
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
```

The server decodes this string, validates the credentials, and grants access if they match a valid user record. After they are decoded, they will look like this:

```
admin:admin
```

The Limitation:
Basic Authentication is considered "weak" because it encodes credentials in Base64, which is a two-way encoding scheme, not encryption. If the connection is not encrypted via HTTPS, an attacker can easily intercept and decode the username and password.

Suggested Alternatives:

1. **JWT (JSON Web Tokens):** More secure as they allow for stateless authentication and can include expiration times.
2. **OAuth2:** The industry standard for delegating access without sharing actual passwords.